Projet Python — Scripting

PRESSE-PAPIER

Un gestionnaire d'historique du presse-papiers dans le terminal

```
$ python presse_papier.py
```

Jérémie BEUGRE | 03 Avril 2026

Problème & Solution

Pourquoi un gestionnaire de presse-papiers personnalisé ?

Limitations actuelles

```
Contexte : Windows | macOS | Linux
```

> *« Le presse-papiers standard est une mémoire volatile et unique : une donnée chasse l'autre, et tout disparaît au redémarrage. »*

Ce projet transforme une fonction éphémère en un véritable outil de productivité durable.

Notre Solution

✓ **Universel & Portable**

Un script unique en ligne de commande compatible Windows, macOS et Linux.

✓ **Sauvegarde Permanente**

Tout est stocké dans un fichier historique.txt qui survit au redémarrage.

✓ **100% Scriptable**

Format texte simple : l'historique est facilement lisible et exploitable par d'autres programmes.

Structure du code

Organisation standard conforme au cours

Organisation du fichier

1. **Imports**
   Bibliothèque standard puis externes
2. **Constantes**
   Déclarées en UPPER_SNAKE_CASE
3. **Fonctions**
   Définitions avec docstrings
4. **Point d'entrée**
   Bloc if __name__ == '__main__':

Principales Fonctions

`sauvegarder(texte)`

Écrit le nouveau contenu dans le fichier historique.

`afficher_historique()`

Lit et formate l'affichage des copies précédentes.

`surveiller()`

Boucle infinie détectant les changements du presse-papiers.

`menu()`

Interface utilisateur simple pour choisir les actions.

Librairie externe : pyperclip

Une interface simple pour interagir avec le presse-papiers du système.

Installation

```
$ pip install pyperclip
```

Seule dépendance du projet

Fonctions Clés

`pyperclip.paste()`

Lire le contenu actuel du presse-papiers du système sous forme de chaîne de caractères.

exemple.py

```python
import pyperclip
# 1. Lire le contenu du presse-papiers
contenu = pyperclip.paste()
```

Fonction : surveiller()

Le cœur du programme : surveillance en temps réel et sauvegarde.

Boucle Infinie

Utilisation de while True pour maintenir le script actif. Une pause de 1 seconde avec time.sleep(1) évite de surcharger le processeur.

Détection de changement

Comparaison entre le contenu actuel et la variable dernier. Si le texte est nouveau et non vide, on déclenche sauvegarder().

Arrêt Propre

Gestion de l'exception KeyboardInterrupt pour permettre à l'utilisateur de quitter proprement avec Ctrl+C.

presse-papier.py

```
def surveiller():
    dernier = ''
    try:
        while True:
            # Lecture du presse-papiers
            contenu = pyperclip.paste()
            if contenu and contenu != dernier:
                dernier = contenu
                sauvegarder(contenu)
            # Pause de 1 seconde
            time.sleep(1)
    except KeyboardInterrupt:
        print('\nArrêt.')
```

Fonction afficher_historique()

Lecture du fichier et reconstruction des blocs de texte.

Format du fichier

```
bonjour
---
hello world
---
```

Note : Le séparateur --- délimite les entrées.

Algorithme

1. Lire toutes les lignes avec readlines().
2. Accumuler les lignes dans une liste temporaire.
3. Afficher le bloc lorsqu'on rencontre ---.

Démonstration

Scénario de test en direct (pas-à-pas)

1 **Lancement**

Exécuter : python clipboard_manager.py

2 **Mode Surveillance**

Choisir l'option 1 pour activer l'écoute.

3 **Copie de données**

Copier plusieurs textes depuis différentes sources.

4 **Confirmation visuelle**

Le terminal affiche : Sauvegardé : '...'

5 **Arrêt**

Interrompre proprement avec Ctrl+C.

6 **Consultation**

Option 2 : afficher l'historique numéroté.

7 **Nettoyage**

Option 3 : vider l'historique et vérifier qu'il est vide.

Conclusion et Perspectives

Bilan du projet et évolutions futures

Ce qui fonctionne

✓ **Historique Permanent**

La persistance des données via historique.txt est fiable.

✓ **Portabilité Totale**

Le script tourne indifféremment sur Windows, macOS et Linux grâce à pyperclip.

✓ **100% Scriptable**

Format texte brut facile à parser par d'autres outils (grep, awk, scripts Python).

Améliorations Possibles

✧ **Interface TUI (Textual)**

Remplacer l'affichage console simple par une interface riche (souris, défilement).

✧ **Recherche & Filtres**

Retrouver rapidement un ancien copié-collé par mots-clés.

✧ **Épinglage**

Garder certains éléments importants en haut de liste (favoris).

✧ **Mode Daemon**

Lancement automatique en arrière-plan au démarrage du système.